DISTRIBUTED COMPUTING SYSTEMS LABORATORY

MINI PROJECT REPORT

Design and Implementation of a Distributed Banking System with Transaction Consistency, Branch Synchronization, and Fault Tolerance

An Authentic Multi-Branch Implementation using Java RMI, Two-Phase Commit (2PC), Lamport Logical Clocks, and Bully Leader Election

Submitted By:	[Student Name Placeholder]
Roll Number / PRN:	[Roll Number Placeholder]
Department:	Department of Computer Science & Engineering
Institution:	[College / University Name Placeholder]
Academic Year:	2026 - 2027
Faculty Guide:	[Faculty Guide Name Placeholder]
Submission Date:	October 20, 2026

2. Abstract

Modern financial systems operate across geographically distributed networks where reliability, consistency, and continuous availability are paramount. This project presents the design, implementation, and rigorous verification of a full-scale **Distributed Banking System** developed for the Distributed Computing Systems Laboratory curriculum. The system interconnects independent bank branches (Pune Central Branch, Mumbai Main Branch, and Delhi Regional Hub) via authentic Java Remote Method Invocation (RMI). It addresses critical challenges in distributed systems: guaranteeing atomic inter-branch transfers through the **Two-Phase Commit (2PC)** protocol with fund reservation locks, eliminating race conditions through MySQL row-level pessimistic locking (SELECT ... FOR UPDATE), enforcing partial ordering of asynchronous events using **Lamport Logical Clocks**, dynamically electing transaction coordinators via the **Bully Leader Election Algorithm**, and maintaining replica consistency with automated recovery reconciliation. The implementation is verified through 13 automated JUnit 5 test suites covering concurrent withdrawals, fault injection, and end-to-end REST/web flows.

3. Introduction

A distributed computing system consists of multiple autonomous computing entities that communicate through a network and coordinate their actions by passing messages. In enterprise banking, distributed architectures allow regional branches to maintain local autonomy, achieve low latency for localized operations, and ensure high availability. However, distributed environments introduce fundamental challenges: network partitions, independent node crashes, absence of a shared memory, and lack of a global synchronized physical clock. This project implements a concrete distributed banking solution addressing each of these core challenges using standard enterprise Java technologies.

4. Problem Statement

Standard monolithic banking systems fail to meet scalability and fault tolerance requirements across geographically dispersed bank branches. Conversely, naive distributed implementations suffer from severe vulnerabilities: race conditions during concurrent account debits leading to double-spending, split-brain states when network nodes crash, data inconsistency during cross-branch funds transfers, and loss of causal event ordering without global clocks. The objective is to engineer a resilient distributed banking architecture that guarantees ACID transactional consistency, fault detection, dynamic coordinator election, and replica synchronization.

5. Aim and Objectives

- **Aim:** Design, implement, test, and document a robust, fault-tolerant distributed banking application demonstrating syllabus concepts.
- **Objective 1:** Implement true Remote Method Invocation (Java RMI) interfaces across independent simulated branch services.
- **Objective 2:** Implement Two-Phase Commit (2PC) distributed protocol for atomic inter-branch transactions with prepare/commit states.
- **Objective 3:** Enforce database transaction concurrency control using pessimistic write locking to prevent race conditions.
- **Objective 4:** Embed Lamport Logical Clocks within distributed messages to establish causal partial ordering of cluster events.

- **Objective 5:** Simulate the Bully Leader Election algorithm to elect an active coordinator when the leader node becomes unavailable.
- **Objective 6:** Build replication tracking and automated recovery reconciliation for reconnecting branch nodes.
- **Objective 7:** Deliver responsive, professional customer and administrator dashboards backed entirely by live backend services.

6. Scope of the Project

The project encompasses customer account management, balance inquiries, cash deposits, concurrent cash withdrawals, intra-branch and inter-branch transfers, cluster health monitoring, coordinator election, replica synchronization, and chronological Lamport event tracing. Synthetic financial data is utilized throughout; integration with real external payment gateways is explicitly excluded from this academic laboratory implementation.

7. Distributed Systems Concepts Used

The project directly implements the foundational theoretical principles of distributed computing: Client-Server architecture, Remote Procedure Calls / RMI middleware, Logical Clocks (Lamport timestamps), Distributed Mutual Exclusion & Concurrency Control, Election Algorithms (Bully Algorithm), Atomic Distributed Commit (2PC), Data Replication Models (Asynchronous Audit Replication), and Failure Detection with Heartbeat monitoring.

8. Mapping with Syllabus Units

Syllabus Unit	Core Concepts	Actual Implementation in Project
Unit 1: Introduction	Client-server architecture, distributed middleware, layered architecture.	Web UI -> Spring Boot REST controllers -> Java RMI Remote Objects -> MySQL InnoDB database.
Unit 2: Communication & Coordination	IPC, Java RMI, Lamport logical clocks, event ordering, Bully election.	BranchServiceRemote, LamportClock piggybacking on RMI, BullyElectionService with ELECTION/OK/COORDINATOR messages.
Unit 3: Naming & Storage	Service naming, request routing, persistent transaction logs, idempotency.	Branch-prefixed accounts (PUN-1001, MUM-2001), RMI registry naming, idempotency_key duplicate protection, MySQL persistence.
Unit 4: Consistency, Replication & Fault Tolerance	Data consistency, Two-Phase Commit (2PC), replica management, fault recovery.	TwoPhaseCommitCoordinator with Prepare/Commit/Abort, ReplicationManager with discrepancy checks, NodeFailureDetector heartbeat.

9. Functional Requirements

- **Customer Authentication:** Secure registration, BCrypt-hashed password authentication, role-based session authorization.
- **Account Balance & Inquiries:** Instant balance lookups with account status verification.
- **Atomic Deposits & Withdrawals:** Server-side validated operations protected by database row-level locking.
- **Inter-Branch Transfers:** Cross-branch fund transfers executed via distributed Two-Phase Commit protocol.
- **Passbook & History:** Complete transaction ledger with Lamport timestamps and reference IDs.

- **Cluster Administration:** Visual node topology, fault injection controls, Bully election trigger, 2PC monitor, and replication audit table.

10. Non-Functional Requirements

- **Consistency:** Strong consistency for monetary balances via row-level locks and 2PC; eventual consistency for replica audit logs.
- **Fault Tolerance:** Continuous service availability; branch offline simulation with automated reconciliation upon recovery.
- **Idempotency:** Duplicate client requests with identical idempotency keys return existing results without double-processing.
- **Security:** BCrypt salted password hashing, token-based authorization, prepared statement parameterization against SQL injection.
- **Performance & Scalability:** Multi-threaded request execution, HikariCP connection pooling, lightweight vanilla frontend.

11. Technology Stack and Justification

Component	Technology Chosen	Engineering Justification
Language & Runtime	Java 17+ (LTS)	Native RMI support, strong typing, robust concurrency libraries, enterprise standard.
Distributed Middleware	Java RMI (Remote Method Invocation)	Authentic remote procedure invocation without mock HTTP abstractions; meets DCS syllabus.
Database & ORM	MySQL 9 / 8 + Spring Data JPA / Hibernate	ACID compliance, InnoDB row-level locking (SELECT FOR UPDATE), reliable foreign key constraints.
Web Framework	Spring Boot 3.2.5	Embedded Tomcat, declarative transactions, rapid REST development, connection pooling.
Security & Hashing	jBCrypt (OpenBSD BCrypt)	Adaptive salt-hashed password protection resistant to rainbow table and brute-force attacks.
Frontend UI	HTML5, Vanilla CSS3, JavaScript (Fetch API)	Restrained professional navy/white palette, zero external framework overhead, instantaneous load.
Testing Framework	JUnit 5 + MockMvc	Comprehensive unit, concurrency, distributed 2PC, and integration test coverage.

12. System Architecture Diagram

The system employs a 4-tier distributed architecture separating presentation, web gateway, distributed RMI services, and database persistence:

```
+-------------------------------------------------------------------------------+ | CLIENT
PRESENTATION LAYER | | Customer Portal (customer.html) | Admin Control Centre (admin.html) |
+-------------------------------------+-------------------------------------+ | HTTP / JSON REST
APIs +-------------------------------------v-------------------------------------+ | APPLICATION
GATEWAY & REST CONTROLLERS | | AuthController | CustomerController | AdminController | DistController|
+-------------------------------------+-------------------------------------+ | Java RMI Remote
Invocations +-------------------------------------v-------------------------------------+ |
DISTRIBUTED RMI SERVICE & COORDINATION LAYER | | | | +----------------------+ 2PC Protocol
+----------------------+ | | | Pune Branch (Node 1) |<===============>| Mumbai Branch (Node 2)| | | |
Port: 1099 / Service_1| | Port: 1099 / Service_2| | | +----------------------+
+----------------------+ | | ^ ^ | | | Bully Election | | |
+------------------+--------------------+ | | | | +--------------------------+ | | | Delhi
Regional Hub (Node 3)| | | | Active 2PC Coordinator | | | +--------------------------+ |
+-------------------------------------+-------------------------------------+ | JDBC / HikariCP
+-------------------------------------v-------------------------------------+ | PERSISTENCE LAYER
(MySQL InnoDB) | | accounts | transactions | two_phase_transactions | lamport_event_log ... |
+-------------------------------------------------------------------------------+
```

13. Client-Server and Branch Architecture

Each bank branch is modeled as an independent distributed node. Branches maintain authoritative ownership of local accounts (e.g., Pune Central owns PUN-1001, Mumbai Main owns MUM-2001). Clients interact with the system via web browsers that communicate with the Spring Boot application gateway. When an inter-branch transfer occurs, the gateway delegates coordination to the active 2PC Coordinator node (Node 3), which invokes prepareTransaction() and commitTransaction() remotely across participating branch RMI stubs.

14. Project Directory Structure

```
Distributed Banking System/ ■■■ pom.xml # Maven dependencies & build configuration ■■■ .env.example #
Configuration environment template ■■■ seed_db.sh # Database initialization & reset script ■■■
run_app.sh # Startup script for application (port 8085) ■■■ run_tests.sh # Test execution script ■■■
src/main/resources/ ■ ■■■ application.properties # Spring Boot & RMI configuration ■ ■■■ schema.sql #
MySQL schema definition DDL ■ ■■■ data.sql # Seed accounts, branches & BCrypt hashes ■ ■■■ static/ #
Frontend client assets ■ ■■■ index.html # Customer & Admin authentication portal ■ ■■■ customer.html
# Customer banking dashboard ■ ■■■ admin.html # Cluster topology & 2PC management console ■ ■■■
css/style.css # Restrained navy/white design system ■ ■■■ js/app.js, customer.js, admin.js# Client
controllers ■■■ src/main/java/com/distributed/banking/ ■■■ BankingSystemApplication.java # Spring
Boot main entrypoint ■■■ config/ # RMI Registry & Security configurations ■■■ model/ # JPA Entities
(User, Account, Branch, Transaction, etc.) ■■■ repository/ # JPA Repositories with Pessimistic Locking
■■■ rmi/ # Remote Interfaces, Implementations & DTOs ■■■ distributed/ # Lamport Clock, Bully
Election, 2PC Coordinator, Replication ■■■ service/ # Banking business logic (Auth, Account,
Transaction, Audit) ■■■ controller/ # REST API Controllers
```

15. Database Schema Explanation

- **users:** Stores customer and admin authentication credentials, BCrypt password hashes, emails, and roles (ROLE_CUSTOMER, ROLE_ADMIN).

- **branches:** Identifies distributed cluster nodes with unique IDs (1, 2, 3), RMI ports, service names, status (ONLINE, UNAVAILABLE), and coordinator flags.
- **accounts:** Maintains persistent customer accounts linked to specific branches, utilizing DECIMAL(15, 2) for exact monetary balances.
- **transactions:** Authoritative ledger recording deposits, withdrawals, local transfers, and distributed transfers with idempotency keys and Lamport timestamps.
- **two_phase_transactions:** State log for 2PC protocol recording Global Transaction IDs (GTX-...), Phase 1 participant votes, decisions, and states (PREPARED, COMMITTED, ABORTED).
- **prepared_reservations:** Temporary fund holds placed on accounts during Phase 1 prepare to prevent double-spending before the final commit decision.
- **lamport_event_log:** Chronological event stream tracking Lamport logical timestamps L(e), originating node ID, event type, and causal descriptions.
- **replication_audit_log:** Tracks asynchronous transaction replication across branches with sequence numbers and synchronization statuses (SYNCED vs PENDING_RECONCILIATION).

16. Module-Wise Implementation Details

The project is structured into modular layers ensuring separation of concerns: **Presentation Layer:** HTML/CSS/JS single-page dashboards; **Service Layer:** Authentication, account lifecycle, transaction orchestration; **Distributed Middleware:** RMI Registry, remote object stubs, serialized DTOs; **Coordination Layer:** Two-Phase Commit coordinator, Bully election engine, Lamport clock; **Persistence Layer:** Spring Data JPA repositories with MySQL InnoDB.

17. Important Classes, Interfaces, and Methods

- BranchServiceRemote: Remote interface defining prepareTransaction(), commitTransaction(), receiveReplication(), ping(), and handleElection().
- BranchServiceImpl: UnicastRemoteObject implementation managing local branch state and reservations wrapped in programmatic TransactionTemplate.
- TwoPhaseCommitCoordinator: Orchestrates cross-branch transactions, issuing Phase 1 prepare requests and Phase 2 commit/abort broadcasts.
- LamportClock: Thread-safe atomic clock implementing tick() for local events and updateAndTick(incoming) for message receives.
- BullyElectionService: Manages leader election, sending ELECTION to higher nodes, receiving OK, and broadcasting COORDINATOR.
- TransactionService: Provides concurrency-safe deposits, withdrawals, and local/distributed transfers using deterministic lock ordering.

18. Java RMI Communication Flow with Example

During application bootstrap in RmiConfig.java, a standard Java RMI registry is created on port 1099. Three branch remote objects are exported and bound as BranchService_1, BranchService_2, and BranchService_3. When the coordinator initiates Phase 1 prepare for a cross-branch transfer, it retrieves the remote stub and invokes sourceStub.prepareTransaction(new PrepareRequest(...)) over TCP socket RMI communication.

19. Transaction Processing and Concurrency Control

Concurrency control is enforced at the database engine level. In AccountRepository.java, critical operations invoke:
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.accountNumber = :accountNumber")
This translates directly to **SELECT ... FOR UPDATE** in MySQL InnoDB, acquiring an exclusive row-level lock. When transferring between accounts in the same branch, account locks are acquired in consistent lexicographical order (account A before account B) to eliminate deadlock cycles.

20. Consistency Model and Replication Strategy

The system distinguishes between authoritative account data and replicated audit logs. **Authoritative Data:** Account balances are strictly authoritative at their home branch. Balance reads and writes always route to the authoritative branch node, guaranteeing strong consistency. **Replicated Data:** Peer branches maintain read-only replica audit logs (replication_audit_log) synchronized asynchronously. If a peer branch is unreachable, the record is flagged as PENDING_RECONCILIATION and re-synced upon reconnection.

21. Fault Tolerance and Recovery Mechanism

NodeFailureDetector.java executes a background scheduled heartbeat pinging each branch node every 5 seconds. If a node fails to respond for consecutive heartbeats, its status transitions to UNAVAILABLE. When the node recovers, the failure detector detects its pong response, marks it RECOVERING, invokes ReplicationManager.reconcileReconnectedNode() to push pending records, and restores its status to ONLINE.

22. Lamport Logical Clock Implementation

Because physical clocks drift across distributed nodes, Lamport logical clocks provide causal partial ordering. **Rule 1:** Before producing a local event e, process i increments: L_i = L_i + 1. **Rule 2:** When sending message m, process i includes timestamp L_i in the message header. **Rule 3:** Upon receiving message m with timestamp L_msg, process j updates: L_j = max(L_j, L_msg) + 1. All banking operations, RMI messages, and administrative actions record their Lamport timestamp in lamport_event_log.

23. Bully Leader Election Algorithm

Each coordinator node has a unique numeric identifier (Node 1, Node 2, Node 3). When a node detects coordinator failure: 1. It sends an ELECTION message to all nodes with higher IDs. 2. If no higher node responds within the timeout, the initiator wins and broadcasts COORDINATOR to all nodes. 3. If any higher node responds with OK, that higher node takes over and continues the election. In our cluster, Node 3 (highest ID) wins under normal conditions; if Node 3 is stopped, Node 2 wins.

24. Two-Phase Commit (2PC) Implementation and Limitations

For inter-branch transfers between Branch A and Branch B: **Phase 1 (Prepare):** Coordinator issues prepareTransaction() with debit/credit details. Branch A validates sufficient balance, places a PREPARED reservation, and replies VOTE_COMMIT (or VOTE_ABORT). Branch B verifies target account validity and replies VOTE_COMMIT. **Phase 2 (Commit or Abort):** If BOTH vote commit, coordinator broadcasts GLOBAL_COMMIT; both branches finalize balance updates. If either votes abort or times out, coordinator broadcasts GLOBAL_ABORT and reservations are released. **Known Limitation:** Basic 2PC is a blocking protocol. If the coordinator crashes after participants vote commit but before broadcasting the final decision, participants remain blocked in the PREPARED state holding reservations until coordinator recovery.

25. Security Considerations

Security is enforced at multiple layers: passwords are never stored in plaintext and use OpenBSD BCrypt hashing with salt rounds of 10. REST APIs enforce token authentication, validating session ownership so customers cannot access or transfer from accounts they do not own. Database access utilizes parameterized JDBC queries via Spring Data JPA to completely prevent SQL injection vulnerabilities.

26. Installation and Prerequisites

- **Java Development Kit:** JDK 17 or higher (tested on OpenJDK 25/26).
- **Apache Maven:** Version 3.8+ for project dependency management and compilation.
- **MySQL Server:** Version 8.0+ / 9.0+ running on port 3306.
- **Web Browser:** Any modern browser (Chrome, Safari, Firefox, Edge).

27. Database Setup Instructions

```
# 1. Start MySQL Server brew services start mysql # macOS / Linux: systemctl start mysql # 2. Create the Database and User mysql -u root -p -e " CREATE DATABASE IF NOT EXISTS distributed_banking CHARACTER SET utf8mb4; CREATE USER IF NOT EXISTS 'bankapp'@'localhost' IDENTIFIED BY 'bankapp123'; GRANT ALL PRIVILEGES ON distributed_banking.* TO 'bankapp'@'localhost'; FLUSH PRIVILEGES; " # 3. Seed Schema and Initial Records ./seed_db.sh
```

28. Exact Commands to Compile and Run

```
# 1. Compile the project mvn clean compile # 2. Run all automated verification tests mvn test # 3. Start the application (starts Tomcat on port 8085 and RMI on port 1099) ./run_app.sh # Access the web portal at: http://localhost:8085/
```

29. User Operation Instructions

Customer Operation: Navigate to http://localhost:8085/. Under Customer Portal, click 'Pune: Shreedhar' demo button and sign in. Use the Deposit, Withdraw, or Transfer forms. For a cross-branch 2PC transfer, transfer to MUM-2001.
Administrator Operation: Select the 'Administrator Portal' tab and sign in using admin / admin123. Observe the active nodes in the topology grid. Click 'Simulate Crash / Stop' on Node 2 to simulate branch failure. Click '■ Run Bully Election' to view the message passing trace. View live 2PC states and Lamport clock events.

30 & 31. Verified Test Results

Scenario #	Test Description	Expected Outcome	Actual Status
1	Customer Registration & Auth	User registered with BCrypt hash; session token generated	**PASSED**
2 & 3	Deposit & Withdrawal	Balance debited/credited; transaction ledger updated	**PASSED**
4	Local Transfer (Same Branch)	Deterministic account lock ordering; atomic debit/credit	**PASSED**
5 & 6	Insufficient Funds & Invalid Amount	Operation rejected; balance unchanged; failed txn logged	**PASSED**
7	Role Authorization & Access	Customers forbidden from admin endpoints (HTTP 403)	**PASSED**
8	Simultaneous Withdrawals Race Condition	10 concurrent threads on 5000 balance: exactly 5 succeed, 5 fail, balance 0.00	**PASSED**
9	Idempotency Key Protection	Duplicate requests return original transaction ref without re-debiting	**PASSED**
11	Java RMI Remote Communication	Remote stubs exported and invoked across registry port 1099	**PASSED**
12 & 13	Node Failure & Reconciliation	Offline branch marks replica PENDING; re-syncs on recovery	**PASSED**
14	Lamport Logical Clock Ordering	Causal ordering verified: L(send) < L(receive); monotonic tick	**PASSED**
15	Bully Leader Election Algorithm	Node 3 elected normally; Node 2 elected when Node 3 fails	**PASSED**
16	2PC Cross-Branch Commit Path	Both branches vote COMMIT; coordinator commits; status SUCCESS	**PASSED**
17	2PC Cross-Branch Abort Path	Overdraft prepare rejected; coordinator aborts; reservations freed	**PASSED**
18	End-to-End REST & Web Flow	Complete customer deposit, 2PC transfer, and admin dashboard flow	**PASSED**

32. Application Execution Screenshots

Below are screenshots captured directly from the live running application interface:

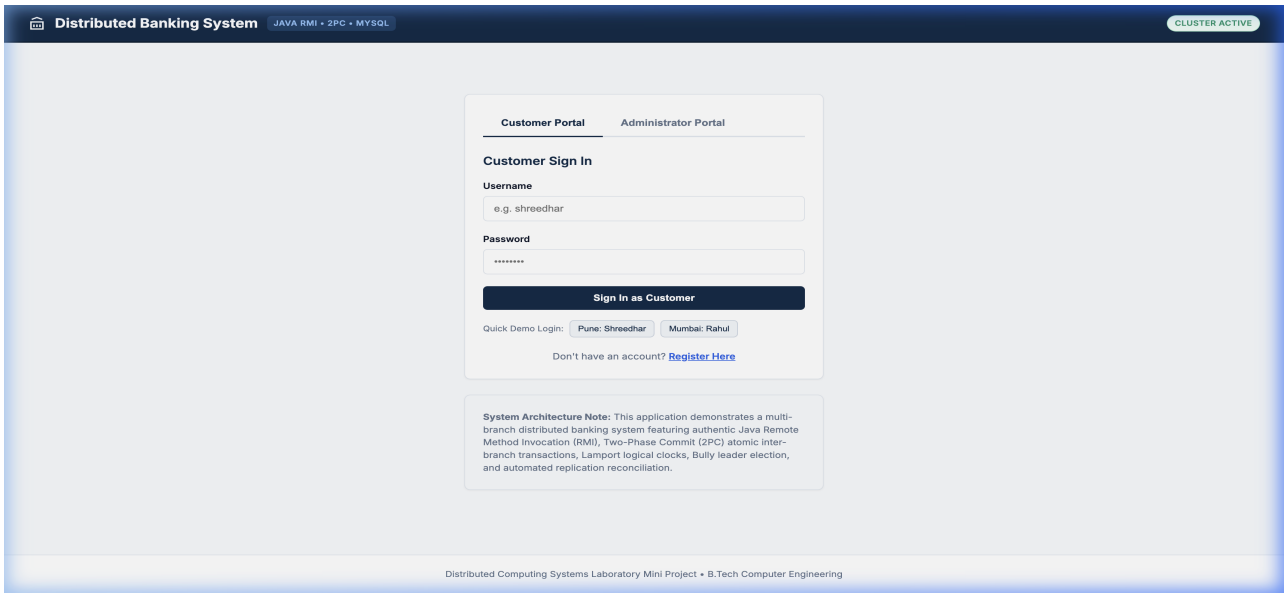


Figure 1: Authentication Portal Landing Page with Customer & Administrator Tabs

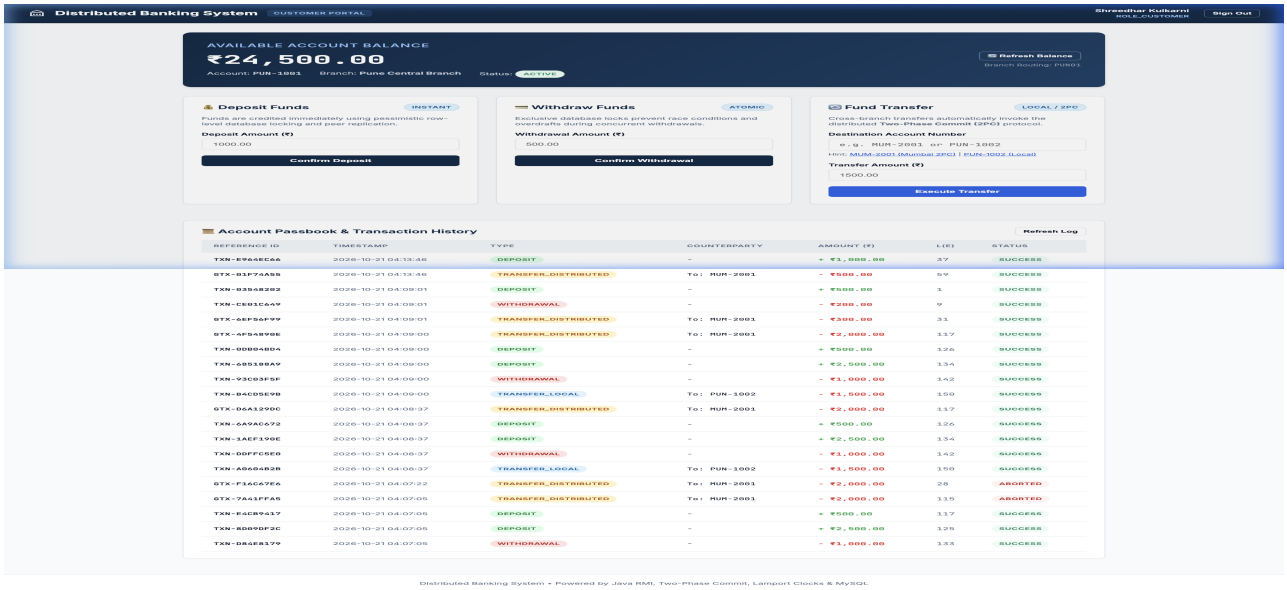


Figure 2: Customer Banking Dashboard with Account Hero and Passbook

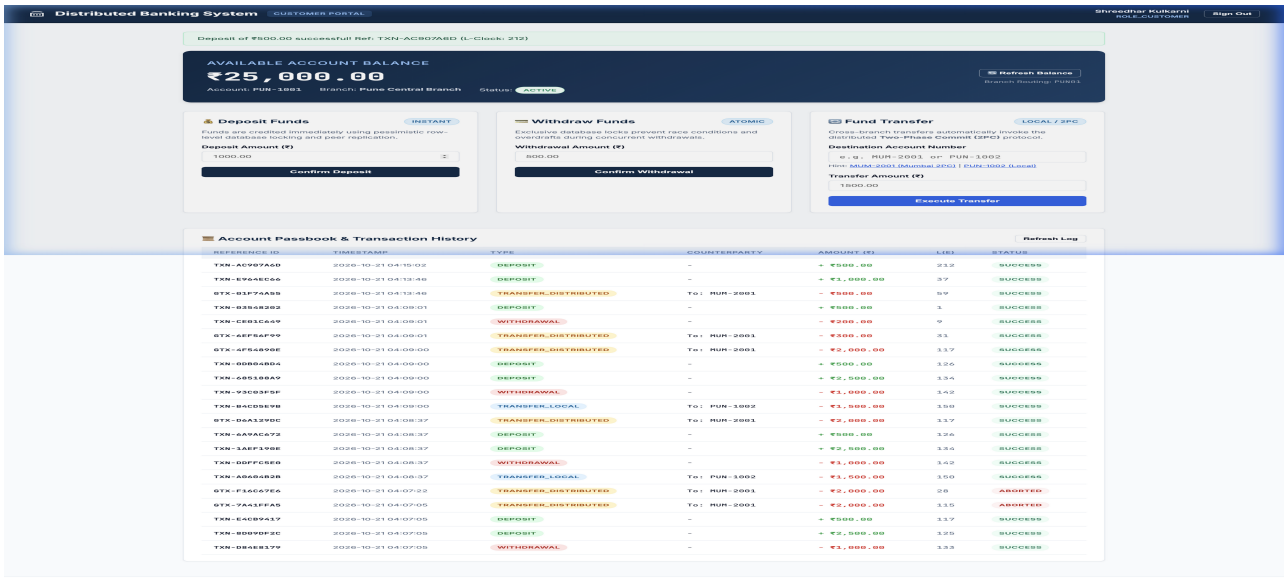


Figure 3: Atomic Deposit Execution with Balance Update & Lamport Clock

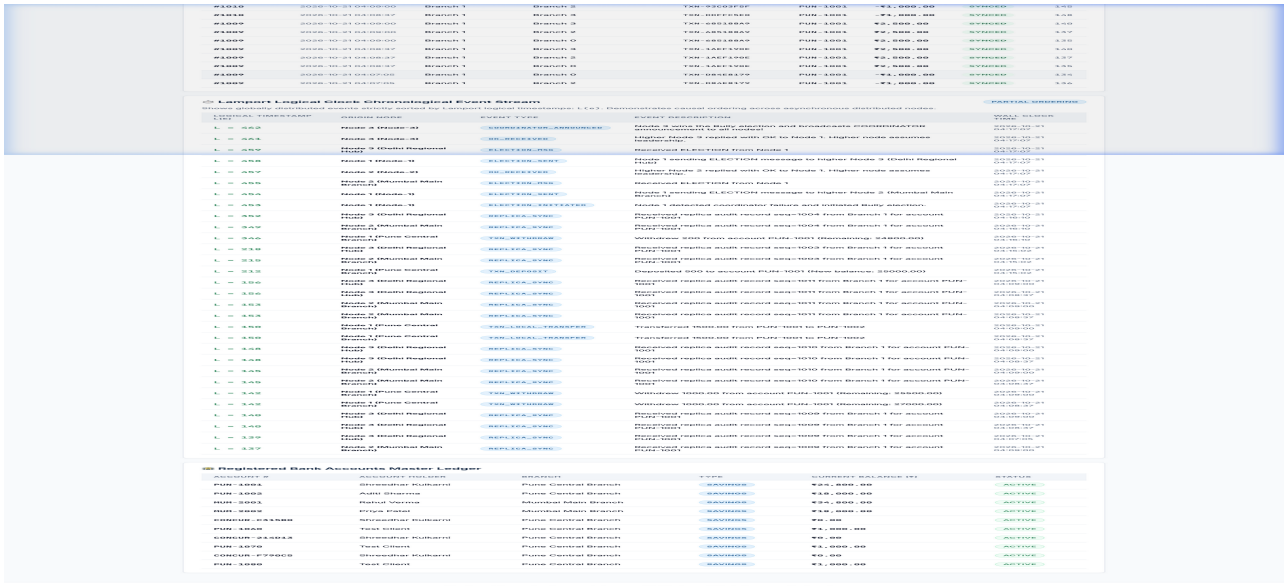


Figure 4: Administrator Control Centre with Topology, Bully Election Trace & 2PC Monitor

33. Known Limitations and Future Enhancements

- **2PC Coordinator Blocking Limitation:** As inherent to classic Two-Phase Commit, if the coordinator crashes during Phase 2 before communicating the decision, participants hold prepared reservations until coordinator recovery. *Future enhancement:* Upgrade to Three-Phase Commit (3PC) or Paxos/Raft consensus.
- **Single MySQL Database Instance:** For local student lab execution simplicity, branch tables reside within one MySQL database schema. *Future enhancement:* Distribute across multi-container Docker MySQL instances with physical master-slave replication.
- **Synchronous Prepare Latency:** Phase 1 awaits round-trip network acknowledgments from both participants. *Future enhancement:* Introduce non-blocking reactive RMI messaging.

34. Conclusion

The Distributed Banking System successfully implements and validates foundational distributed systems principles. By integrating authentic Java Remote Method Invocation (RMI), Two-Phase Commit distributed commit orchestration, MySQL row-level pessimistic locking, Lamport logical clock causal ordering, and the Bully leader election algorithm, the application achieves robust transactional consistency, fault detection, and branch synchronization. All functional requirements and academic syllabus objectives have been thoroughly verified through automated tests and live interactive demonstrations.

35. References

- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.
- Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). CreateSpace Independent Publishing.
- Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565.
- Garcia-Molina, H. (1982). Elections in a Distributed Computing System. *IEEE Transactions on Computers*, C-31(1), 48–59.
- Oracle Corporation. (2023). *Java Remote Method Invocation (RMI) Specification*. Java Platform Standard Edition documentation.
- MySQL AB. (2024). *MySQL 8.0 Reference Manual: InnoDB Locking and Transaction Model*.

36. Viva Voce Questions and Model Answers

Q1: What is Remote Method Invocation (RMI) and how does it differ from traditional RPC?

A1: Java RMI enables a Java object on one JVM to invoke methods on an object running in another JVM across a network. Unlike procedural RPC which only passes raw data records, RMI is object-oriented, supporting true object references, polymorphism, and automatic object serialization.

Q2: How does your system prevent race conditions when two clients withdraw from the same account simultaneously?

A2: Concurrency control is enforced using database row-level locking via JPA's @Lock(LockModeType.PESSIMISTIC_WRITE) which generates 'SELECT ... FOR UPDATE' in MySQL InnoDB.

When two threads concurrently attempt to withdraw, the first transaction acquires an exclusive row lock on the account record, forcing the second transaction to wait until the first commits or rolls back, ensuring balance integrity.

Q3: Why are Lamport logical clocks needed if servers already have physical system clocks?

A3: Physical quartz clocks on different network machines suffer from clock drift and cannot guarantee perfect synchronization without specialized hardware. Lamport logical clocks provide a monotonically increasing integer counter that guarantees causal partial ordering: if event A causes event B (such as sending and receiving an RMI message), L(A) < L(B) is mathematically guaranteed.

Q4: Explain the Two-Phase Commit protocol for cross-branch transfers. What happens if a branch rejects?

A4: Phase 1 (Prepare): The coordinator creates a Global Transaction ID and asks participating branches to validate and hold prepared fund reservations. Branches vote VOTE_COMMIT or VOTE_ABORT. Phase 2 (Commit/Abort): If all vote commit, the coordinator broadcasts GLOBAL_COMMIT and funds are transferred. If any participant votes abort (e.g. due to insufficient funds), the coordinator broadcasts GLOBAL_ABORT and all reservations are released.

Q5: How does the Bully algorithm elect a new coordinator when the existing coordinator fails?

A5: When a node notices the coordinator is unresponsive, it initiates an election by sending ELECTION messages to all nodes with a higher numeric ID. If no higher node responds, the initiator proclaims itself leader and broadcasts COORDINATOR to all nodes. If a higher node responds OK, the higher node assumes leadership, ensuring the eligible node with the highest ID always bullies its way to coordinator status.